

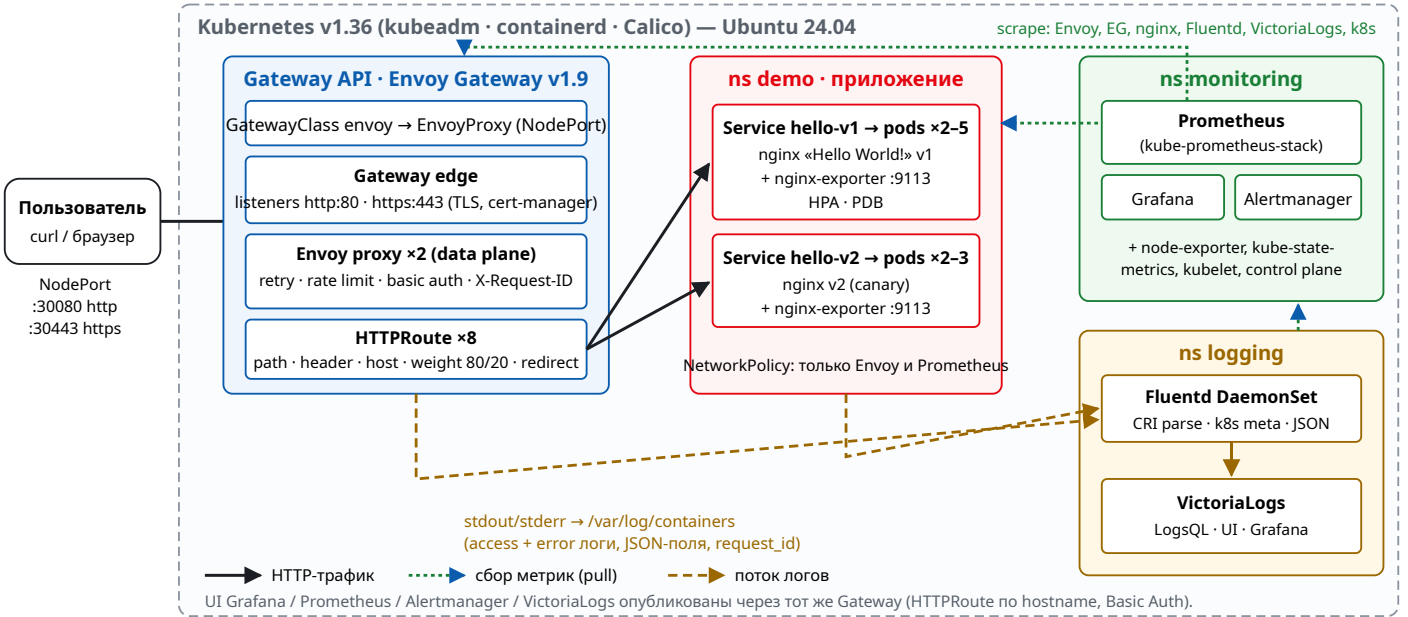
Паспорт решения · MTS Engineer Hack — DevOps

Автор: Схрейдер · Репозиторий: <https://github.com/krantro2938/mts-devops-k8s> (ветка main) · Kubernetes на kubeadm + Gateway API + Prometheus + Fluentd, развёртывание и проверка — make up

1. Архитектура и состав решения

Версия Kubernetes	v1.36.5 (kubeadm / kubelet / kubectl), containerd 2.2.9 + runc 1.4.3, CNI Calico v3.33
Способ развёртывания Kubernetes	kubeadm (kubeadm init с конфигом v1beta4) — скрипт scripts/bootstrap-node.sh: подготовка Ubuntu (swap, модули, sysctl), установка containerd/kubeadm, init, kubeconfig. Одноузловой кластер, поддерживаю добавление worker-узлов.
Реализация Gateway API	Envoy Gateway v1.9.2 (Envoy 1.39), Gateway API v1.6.1 standard channel; ресурсы GatewayClass, Gateway, HTTPRoute ×8 + политики Envoy Gateway
Инструменты автоматизации	bash-скрипты (идемпотентные), Helm 4 (все компоненты + собственный chart приложения), Kustomize, Makefile; GitHub Actions CI (lint + e2e на Ubuntu 24.04); все версии — в versions.env
Инструмент логирования	Fluentd v1.19.3 (DaemonSet, Helm chart fluent/fluentd) → хранилище VictoriaLogs v1.53 (LogsQL, Web UI, Grafana)
Способ развёртывания Prometheus	Helm chart kube-prometheus-stack 91.8.2 (Prometheus Operator v0.94): Prometheus, Alertmanager, Grafana 13, node-exporter, kube-state-metrics; цели — PodMonitor/ServiceMonitor, правила — PrometheusRule
ОС, на которой тестировалось	Ubuntu 24.04 LTS (x86_64): чистая ВМ GitHub Actions ubuntu-24.04 — полный цикл kubeadm → deploy → smoke-тесты → повторный deploy → smoke-тесты
Приложение	nginx 1.30 (nginx-unprivileged, конфиг в ConfigMap) — версии v1 и v2, ответ Hello World!, JSON access-лог, sidecar nginx-prometheus-exporter

Архитектурная схема



Как запустить: на чистой Ubuntu 24.04 (4 vCPU / 8 GB): `git clone .../mts-devops-k8s && cd mts-devops-k8s && make up` (или по шагам: `sudo ./scripts/bootstrap-node.sh → ./scripts/deploy.sh → ./scripts/smoke-test.sh`). Проверка: `curl http://<IP>:30080/ → Hello World!` . ~10-15 минут, без ручного создания ресурсов.

2. Реализованный функционал — обязательные пункты

Пункт	Как реализовано технически	Обоснование выбора	Как проверить
1. Kubernetes-окружение	<code>bootstrap-node.sh</code> : <code>swap off</code> , <code>br_netfilter/overlay</code> , <code>sysctl</code> ; <code>containerd 2.2 + runc (sha256)</code> , <code>kubeadm/kubelet/kubectl 1.36.5</code> из <code>pkgs.k8s.io (apt hold)</code> ; <code>kubeadm init</code> с конфигом (<code>systemd cgroup</code> , <code>metrics-адреса control plane</code> открыты для <code>Prometheus</code>); <code>Calico CNI</code> ; <code>local-path StorageClass</code> ; <code>metrics-server</code>	<code>kubeadm</code> — приоритетный, вендор-независимый путь; <code>Calico</code> даёт <code>NetworkPolicy</code> ; без облачных сервисов	<code>kubectl get nodes -o wide</code> → <code>Ready</code> , <code>v1.36.5</code> , <code>Ubuntu 24.04</code> , <code>containerd 2.2.9</code>
2. Веб-приложение	Собственный <code>Helm-chart deploy/app/hello</code> : <code>nginx-unprivileged</code> (публичный образ), конфиг из <code>ConfigMap (envsubst)</code> , 2 версии (<code>v1/v2</code>), <code>Service</code> , <code>HPA</code> , <code>PDB</code> , пробы; <code>access-лог</code> в <code>JSON (stdout)</code> , <code>error-лог (stderr)</code> ; <code>sidecar nginx-prometheus-exporter</code>	Свое приложение не требуется — достаточно конфигурации; <code>JSON-лог</code> удобно разбирать в поля	<code>curl http://IP:30080/</code> → <code>Hello World!</code> + версия и под; <code>/nope</code> → <code>404</code> , <code>/error</code> → <code>500</code>
3. Gateway API	<code>Gateway API CRDs v1.6.1 + Envoy Gateway v1.9.2 (Helm, CRD — server-side apply)</code> . <code>GatewayClass envoy</code> → <code>EnvoyProxy (NodePort 30080/30443, 2 реплики, PDB)</code> ; <code>Gateway edge (ns gateway, listeners http/https, allowedRoutes no label namespace)</code> ; <code>HTTPRoute hello</code> → <code>Service hello-v1/v2</code>	<code>Envoy Gateway</code> — CNCF-проект «Gateway API first», полный <code>standard channel</code> и политики через <code>Policy Attachment</code> , без замены <code>CNI / service mesh</code>	<code>kubectl get gatewayclass,gateway,httproute -A (Accepted / Programmed)</code> ; <code>curl -i http://IP:30080/</code> → заголовок <code>x-served-by: envoy-gateway</code>
4. Мониторинг	<code>kube-prometheus-stack</code> ; <code>PodMonitor</code> для <code>nginx-exporter</code> , <code>Envoy proxy</code> , <code>Envoy Gateway</code> , <code>Calico</code> ; <code>ServiceMonitor</code> для <code>Fluentd</code> , <code>VictoriaLogs</code> , <code>cert-manager</code> , <code>metrics-server</code> ; все цели <code>control plane kubeadm (etcd, scheduler, controller-manager, kube-proxy)</code> — <code>UP</code>	<code>Operator-подход</code> : цель описывается рядом с компонентом декларативно; <code>Grafana</code> и <code>Alertmanager</code> в комплекте	<code>kubectl get --raw '/api/v1/namespaces/monitoring/services/kps-prometheus:9090/proxy/api/v1/query?query=up'</code> ; <code>UI prometheus.demo.local:30080</code> → <code>Targets</code> ; запросы — <code>README §6</code>
5. Логирование	<code>Fluentd DaemonSet</code> (официальный <code>chart/образ</code>): <code>tail /var/log/containers (CRI)</code> → <code>concat</code> → <code>kubernetes_metadata</code> → разбор <code>JSON access-логов nginx</code> и <code>Envoy</code> → <code>log_type access/error</code> → <code>out_http (JSON lines, дисковый буфер, retry)</code> → <code>VictoriaLogs</code>	<code>Fluentd</code> — гибкий пайплайн без сборки своего образа; <code>VictoriaLogs</code> — полнотекстовый поиск при ~100 MB RAM (<code>ES/OpenSearch</code> не помещаются на 1 узел)	запрос с уникальным <code>X-Request-ID</code> , затем <code>LogsQL</code> -запрос через <code>API (README §7)</code> → запись <code>log_type=access status=200</code> ; <code>UI logs.demo.local:30080</code> , <code>Grafana Explore</code>
6. Ubuntu 24.04	Скрипты проверяют ОС; <code>CI-задача e2e</code> выполняет весь сценарий на чистой <code>VM ubuntu-24.04</code>	Доказуемая воспроизводимость на целевой ОС при каждом коммите	вкладка <code>Actions</code> репозитория, задача <code>e2e</code>
7. Автоматизация	3 команды или <code>make up</code> ; <code>helm upgrade --install</code> , <code>kubectl apply --server-side</code> , проверки состояния в <code>bootstrap</code> ; пароли генерируются один раз и хранятся в <code>Secret</code>	Минимум зависимостей у эксперта (<code>bash</code> , <code>curl</code> , <code>git</code>); идемпотентность по построению	повторный запуск <code>deploy.sh / bootstrap-node.sh</code> → <code>smoke-тесты</code> снова зелёные (так делает <code>CI</code>)
8. Документация	<code>README</code> : архитектура (<code>Mermaid</code>), версии, требования, шаги, проверки, ограничения; <code>docs/DECISIONS.md</code>	Эксперт может проверить каждый пункт копированием команд	<code>README.md</code>

2.1 Дополнительные улучшения

Улучшение	Что и как реализовано	Обоснование	Как проверить
Расширенный Gateway API	Маршрутизация по пути (/v2 + URLRewrite), заголовку (X-Version: v2), hostname (v2.demo.local); traffic splitting 80/20 (canary.demo.local); HTTPS TLS Terminate с сертификатом cert-manager (свой CA); редирект HTTP→HTTPS; ResponseHeaderModifier; timeouts; несколько backend; UI мониторинга/логов как отдельные HTTPRoute из своих namespace	Показывает реальные сценарии: canary-релиз, версионирование API, мульти-тенантный общий Gateway	таблица в README §5; smoke-test.sh
Политики Envoy Gateway	BackendTrafficPolicy: retry (connect-failure/reset/502-504) + circuit breaker; local rate limit 10 rps → 429; SecurityPolicy: Basic Auth на Prometheus/Alertmanager/VictoriaLogs; ClientTrafficPolicy: сквозной X-Request-ID , TLS ≥ 1.2	Надёжность и защита на уровне шлюза без изменения приложения	burst-запросы к canary → 429; UI без пароля → 401
CI/CD	GitHub Actions: lint — shellcheck, yamllint, helm lint, kubeconform всех манифестов и отрендеренных чартов против схем CRD (Gateway API, Envoy Gateway, Prometheus Operator, cert-manager); e2e — kubectl на чистой Ubuntu 24.04 → deploy → 42 smoke-проверки → повторный deploy → smoke; диагностика артефактом при сбое	Каждое изменение доказуемо работает и идиempотентно	Actions → workflow ci; локально make lint, make test
Расширенный мониторинг	HTTP-метрики Gateway: RPS , коды ответов , latency p50/p95/p99 , 429; nginx; CPU/RAM подов; Fluentd; recording rules; 8 алертов (5xx, latency, недоступность, Fluentd, VictoriaLogs) → Alertmanager; дашборд Grafana «MTS Demo» (метрики + живые логи)	Golden signals для сервиса и наблюдаемость самой системы логирования	make traffic, Grafana grafana.demo.local:30080 (make creds)
Централизованные логи с поиском	Разбор JSON в поля (status, uri, request_id, request_time...), классификация access/error, логи Gateway, корреляция по X-Request-ID, хранение 7 дней, поиск LogsQL, UI и Grafana	Логи пригодны для расследований и аналитики, а не только «сырые строки»	README §7 (запрос по request_id находит запись и в nginx, и в Envoy)
Надёжность и безопасность	≥2 реплики, HPA, PDB, пробы, preStop, rolling update без простоя; NetworkPolicy default-deny; Pod Security Admission (restricted); non-root, read-only rootfs, drop ALL, seccomp; случайные пароли только в Secret; sha256 скачиваемых бинарников; закреплённые версии	Базовые production-практики	kubectl get hpa,pdb,netpol -A; kubectl get ns --show-labels

3. Ревью работы и потенциальное масштабирование

Главная особенность решения

Доказуемая воспроизводимость: весь путь от «голой» Ubuntu 24.04 до работающего кластера на kubeadm с Gateway API, метриками и логами выполняется скриптами и на каждом коммите прогоняется в CI на чистой VM дважды подряд, с 42 автоматическими проверками каждого требования. Наблюдаемость сквозная: один запрос с X-Request-ID виден в access-логе Envoy и nginx, а его код ответа и задержка — в метриках Gateway в Prometheus.

Самое сложное решение

Выбор хранилища логов для Fluentd: Elasticsearch/OpenSearch (стандарт, но $\geq 1\text{--}2$ GB RAM на JVM), Loki (лёгкий, но в официальном образе Fluentd нет плагина — пришлось бы собирать и публиковать свой образ) или VictoriaLogs. Выбран VictoriaLogs: укладывается в ~ 100 MB RAM на одном узле, принимает JSON lines встроенным HTTP-выводом Fluentd (официальный образ без доработок), даёт полнотекстовый поиск по всем полям, UI и интеграцию с Grafana. Второе непростое решение — публикация Gateway через фиксированные NodePort вместо MetalLB, чтобы не зависеть от сети стенда эксперта.

Предложения по дальнейшему развитию

Что добавить	Что для этого нужно
HA control plane (3 узла, kube-vip) и MetalLB / BGP -анонс адреса Gateway в сеть оператора вместо NodePort	3+ VM, VIP-адрес; для BGP — пиринг с маршрутизаторами (сетевая команда)
GitOps (Argo CD / Flux): кластер сам применяет состояние из Git, промоушен dev → stage → prod	доступ кластера к Git; отдельные окружения
Телеком-протоколы через Gateway API: GRPCRoute (HTTP/2, 5G SBA), TCPRoute/UDPRoute (SIP, Diameter, RADIUS), TLSRoute passthrough	experimental channel Gateway API; тестовые CNF/стенды сигнализации
Глобальный rate limit (на абонента/ключ API) и OIDC/JWT -аутентификация на Gateway	Redis для Envoy Rate Limit Service; IdP (Keycloak / корпоративный SSO)
Долгосрочное хранение и масштабирование: VictoriaMetrics/Thanos для метрик, VictoriaLogs cluster , архив в S3; требования регуляторов к срокам хранения логов	S3-совместимое объектное хранилище (MinIO / Ceph RGW), диски под retention
Трейсинг : OpenTelemetry Collector + Jaeger/Tempo, трассировка из Envoy	доп. ресурсы под хранилище трасс
Supply-chain security : Trivy-сканирование, подпись образов (cosign), политики Kyverno, внешние секреты (Vault + ESO)	Vault/секрет-хранилище, приватный registry
Air-gapped установка для изолированных площадок (edge/MEC): зеркало образов и чартов	Harbor / приватный registry и Helm-репозиторий на площадке
Нагрузка CNF: Multus + SR-IOV / DPDK , CPU pinning и HugePages (Topology Manager)	серверы с SR-IOV NIC, настройка BIOS/ядра
Интеграция алертов с NOC (Telegram / e-mail / ServiceDesk), SLO-алерты по error budget	каналы/токены уведомлений (через Secret, не в git)

Известные ограничения (подробно — README §10)

- Один control-plane узел; публикация через NodePort 30080/30443; хранилище local-path (данные на диске узла, без репликации).
- TLS на собственном CA для demo.local (для curl — --cacert); metrics-server с --kubelet-insecure-tls; Fluentd работает от root (только DAC_READ_SEARCH).